

Java

Esami

Esame 4/9/2018:

1. *RandomSingleton* per creare un *RandomIterator*
2. *funzionemultiFact()* che data una *Collection<Integer>* produce una *Collection<FactThread>*, in modo che per ogni intero in input viene eseguito lo spawn di un nuovo thread che ne computa il fattoriale e ne conserva il risultato.

```
1 public static class RandomSingleton {
2     private RandomSingleton() {}
3     @Nullable
4     private static Random instance;
5     @NotNull
6     private static Random instance() {
7         if (instance == null)
8             instance = new Random();
9         return instance;
10    }
11    // fare un setter per cambiare il seed del PRNG e' uno dei modi possibili per riprodurre il comportamento
12    // originale della classe Random del JDK.
13    // normalmente, per avere un seed specifico    necessario costruire un oggetto Random tramite l'apposito
14    // costruttore.
15    // nella nostra conversione a singleton rappresentiamo invece questo aspetto come un setter che internamente
16    // reistanzia il singleton in maniera trasparente all'utente.
17    public void setSeed(int seed) {instance = new Random(seed);}
18    public static int nextInt() {return instance().nextInt();}
19    public static boolean nextBoolean() {return instance().nextBoolean();}
20    public static double nextDouble() {return instance().nextDouble();}
21 }
22 public static class RandomIterator implements Iterator<Integer> {
23     private int len;
24     public RandomIterator(int len) {this.len = len;}
25     @Override
26     public boolean hasNext() {return len > 0;}
27     @Override
28     public Integer next() {
29         --len;
30         return RandomSingleton.nextInt();
31     }
32 }
```

```
1 public static class FactThread extends Thread {
2     private int n, result;
3     public FactThread(int n) {this.n = n;}
4     private static int fact(int n) {return n < 2 ? 1 : n * fact(n - 1);}
5     @Override
6     public void run() {result = fact(n);}
7     public int getResult() {return result;}
8 }
9 public static Collection<FactThread> multiFact(Collection<Integer> ns) {
10     Collection<FactThread> r = new ArrayList<>();
11     for (int n : ns) {
12         FactThread t = new FactThread(n);
13         t.start();
14         r.add(t);
15     }
16     return r;
17 }
18 public static void main(String[] args) {
19     for (FactThread t : multiFact(Arrays.asList(1, 2, 3, 4, 5, 6))) {
20         try {
21             t.join();
22         } catch (InterruptedException e) {
23             e.printStackTrace();
24         }
25         System.out.println(String.format("%s_ %d", t, t.getResult()));
26     }
27 }
```

Esame 30/5/2019:

1. Costruire *Sequenza di Fibonacci* con sistema di *caching*

```
1 public static class FiboSequence implements Iterable<Integer> {
2     private final int max;
3     private final Map<Integer, Integer> cache = new HashMap<>();
4     public FiboSequence(int max) {this.max = max;}
5     @Override
6     public Iterator<Integer> iterator() {
7         return new Iterator<>() {
8             private int i = 0;
9             @Override
10            public boolean hasNext() {return i < max;}
11            @Override
12            public Integer next() {return fib(i++);}
13            private int fib(int n) {
14                if (n < 2) return 1;
15                else {
```

```

16         Integer x = cache.get(n);
17         if (x != null) return x;
18         else {
19             int r = fib(n - 1) + fib(n - 2);
20             cache.put(n, r); // si commenti questo statement per disabilitare la cache e vedere la differenza
                               enorme di tempi di calcolo
21             return r;
22         }
23     }
24 }
25 };
26 }
27 }
28 public static class GlobalFiboSequence implements Iterable<Integer> {
29     private final int max;
30     private final static Map<Integer, Integer> cache = new HashMap<>(); // tutto uguale a FiboSequence eccetto per la
                               cache che statica
31     public GlobalFiboSequence(int max) {this.max = max;}
32     @Override
33     public Iterator<Integer> iterator() {
34         return new Iterator<>() {
35             private int i = 0;
36             @Override
37             public boolean hasNext() {return i < max;}
38             @Override
39             public Integer next() {return fib(i++);}
40             private int fib(int n) {
41                 if (n < 2) return 1;
42                 else {
43                     Integer x = cache.get(n);
44                     if (x != null) return x;
45                     else {
46                         int r = fib(n - 1) + fib(n - 2);
47                         cache.put(n, r); // si commenti questo statement per disabilitare la cache e vedere la
                                         differenza enorme di tempi di calcolo
48                         return r;
49                     }
50                 }
51             }
52         };
53     }
54 }

```

Tree

Esame 1/6/2023:

1. inserire in maniera ricorsiva un nodo rispettando la proprietà degli alberi usando un *Comparator*
2. implementare interfaccia *iterabile* per scorrere *in-order (dfs)* l'albero tramite utilizzo di Collection
3. trovare *min* e *max* nell'albero utilizzando con complessità $O(\log_2(n))$
4. definire sottoclasse dell'albero con tipo T vincolato a essere sottotipo di *Comparable* *<?superT >* che sposta logica sul *type parameter*. Il costruttore chiama super tramite lambda.

```

1 public static class BST<T> implements Iterable<T> {
2     @NotNull protected final Comparator<? super T> cmp;
3     @Nullable protected Node root;
4     public BST(@NotNull Comparator<? super T> cmp) { this.cmp = cmp; }
5     public void insert(@NotNull T x) { root = insertRec(root, x); }
6     // ----- 1 -----
7     @NotNull
8     protected Node insertRec(@Nullable Node n, @NotNull T x) {
9         if (n == null)
10             return new Node(x);
11         int r = cmp.compare(x, n.data);
12         if (r < 0)
13             n.left = insertRec(n.left, x);
14         else if (r > 0)
15             n.right = insertRec(n.right, x);
16         return n;
17     }
18     // ----- 2 -----
19     protected void dfsInOrder(@Nullable Node n, @NotNull Collection<T> out){
20         if (n != null) {
21             dfsInOrder(n.left, out);
22             out.add(n.data);
23             dfsInOrder(n.right, out);
24         }
25     }
26     @NotNull
27     @Override
28     public Iterator<T> iterator() {
29         Collection<T> c = new ArrayList<>();
30         dfsInOrder(root, c);
31         return c.iterator();
32     }
33     // ----- 3 -----
34     @Nullable
35     public T min() {
36         if (root == null) return null;

```

```

37     Node n = root;
38     while(n.left != null) n = n.left;
39     return n.data;
40 }
41 @Nullable
42 public T max() {
43     if (root == null) return null;
44     Node n = root;
45     while(n.right != null) n = n.right;
46     return n.data;
47 }
48 protected class Node {
49     @NotNull private final T data;
50     @Nullable protected Node left, right;
51     protected Node(@NotNull T data, @Nullable Node left, @Nullable Node right) {
52         this.data = data;
53         this.left = left;
54         this.right = right;
55     }
56     protected Node(@NotNull T data) {
57         this(data, null, null);
58     }
59 }
60 @Override
61 @NotNull
62 public String toString() {
63     StringBuilder sb = new StringBuilder();
64     for (T x : this) {
65         sb.append(x);
66         sb.append(" ");
67     }
68     return sb.toString();
69 }
70 }
71 // ----- 4 -----
72 public static class BST2<T extends Comparable<? super T>> extends BST<T> {
73     public BST2() {
74         super(Comparable::compareTo);
75     }
76 }

```

Esame 13/9/2022:

1. costruttori o pseudo costruttori ovvero metodi statici ricorsivi per creare alberi facili da costruire e non modificabili
2. override del metodo *equals* per permettere il confronto tramite due alberi
3. override del metodo *toString* pretty printer
4. implementare interfaccia *iterabile* per scorrere *in-order (dfs)* l'albero tramite usilio di Collection
5. implementare main per *testare* i punti precedenti

```

1 public static class TreeNode<T> implements Iterable<T> {
2     @NotNull private final T data;
3     @Nullable private final TreeNode<T> left, right;
4     @Nullable private TreeNode<T> parent = null;
5     // ----- 1 -----
6     public TreeNode(@NotNull T data, @Nullable TreeNode<T> left, @Nullable TreeNode<T> right) {
7         this.data = data;
8         this.left = left;
9         this.right = right;
10        if (left != null) left.parent = this;
11        if (right != null) right.parent = this;
12    }
13    public static <T> TreeNode<T> l(@NotNull T data, @NotNull TreeNode<T> left) {
14        return new TreeNode<>(data, left, null);
15    }
16    public static <T> TreeNode<T> r(@NotNull T data, @NotNull TreeNode<T> right) {
17        return new TreeNode<>(data, null, right);
18    }
19    public static <T> TreeNode<T> lr(@NotNull T data, @NotNull TreeNode<T> left, @NotNull TreeNode<T> right) {
20        return new TreeNode<>(data, left, right);
21    }
22    public static <T> TreeNode<T> v(@NotNull T data) {
23        return new TreeNode<>(data, null, null);
24    }
25    // ----- 2 -----
26    @Override
27    public boolean equals(@Nullable Object o) {
28        if (o instanceof TreeNode) {
29            BlockingQueue<Integer> b;
30            TreeNode<T> t = (TreeNode<T>) o;
31            return areEqual(data, t.data) && areEqual(left, t.left) && areEqual(right, t.right);
32        }
33        return false;
34    }
35    private static boolean areEqual(@Nullable Object a, @Nullable Object b) {
36        return a == b || (a != null && a.equals(b)); //oppure return Objects.equals(a, b);
37    }
38    // ----- 3 -----
39    @Override
40    @NotNull
41    public String toString() {

```

```

42         return String.format("%s%s", data, left != null ? String.format("(%s)", left) : "", right != null ?
43             String.format("[%s]", right) : "");
44     }
45     // ----- 4 -----
46     @Override
47     private Iterator<T> iterator() {
48         Collection<T> r = new ArrayList<>();
49         dfs(r);
50         return r.iterator();
51     }
52     private void dfs(Collection<T> c) {
53         c.add(data);
54         if (left != null) left.dfs(c);
55         if (right != null) right.dfs(c);
56     }
57 }
58 // ----- 5 -----
59 TreeNode<Integer> t1 = TreeNode.lr(1,TreeNode.lr(2,TreeNode.v(3),TreeNode.v(4)),null);
60 TreeNode<Integer> t2 = TreeNode.lr(1,TreeNode.r(5,TreeNode.lr(6,TreeNode.v(7),null)),null);
61 // test pretty printer
62 System.out.println("pretty printer:");
63 System.out.println("t1:␣" + t1);
64 System.out.println("t2:␣" + t2);
65 // test equality
66 System.out.print("equality:␣");
67 System.out.println(t1.equals(t2) + ",␣" + (t1.left != null ? t1.left.equals(t2.right) : ""));
68 // test iterator
69 System.out.print("iterator:␣");
70 for (Integer n : t1) System.out.printf("%d␣", n);

```

Pool

Esame 13/1/2023:

1. Creare classe *SimplePool* che implementa *Pool* e realizza un pool usando *LinkedBlockingQueue*. Un container che acquisisce unna risorsa e la restituisce
2. Creare classe *AutoPool* parametrica su T e implementa *Pool < Resource < T >>* al fine di usarlo come proxy.

```

1 public interface Pool<R> {
2     R acquire() throws InterruptedException;
3     void release(R x);
4 }
5 // ----- 2 -----
6 public static class SimplePool<T> implements Pool<T> {
7     private final BlockingQueue<T> q = new LinkedBlockingDeque<>();
8     @Override
9     public T acquire() throws InterruptedException { return q.take(); }
10    @Override
11    public void release(T x) {q.add(x);}
12 }
13 public interface Resource<T> {
14     T get();
15     void release();
16 }
17 public static class AutoPool<T> implements Pool<Resource<T>> {
18     private final BlockingQueue<T> q = new LinkedBlockingDeque<>();
19     public void add(T x) {q.add(x);}
20     public Resource<T> acquire() throws InterruptedException {
21         T r = q.take();
22         return new Resource<>() {
23             @Override
24             public T get() {
25                 System.out.printf("acquired:␣%s%n", r);
26                 return r;
27             }
28             @Override
29             public void release() {
30                 System.out.printf("released:␣%s%n", r);
31                 add(r);
32             }
33             @SuppressWarnings("deprecation")
34             @Override
35             protected void finalize() {release();}
36         };
37     }
38     @Override
39     public void release(Resource<T> x) {x.release();}
40 }
41 public static void main(String[] args) {
42     AutoPool<Integer> pool = new AutoPool<>();
43     Random rnd = new Random();
44     for (int i = 0; i < 5; ++i) {pool.add(i);}
45     try {
46         while (true) {
47             Resource<Integer> r = pool.acquire();
48             System.out.println("using␣" + r.get());
49             Thread.sleep(Math.abs(rnd.nextInt() % 1000));
50         }
51     } catch (InterruptedException e) {e.printStackTrace();}
52 }

```

Esame 22/1/2019:

1. Definire la classe *BasicPool* generica su un tipo *T*
2. Implementare classe *SimplePool* che implementa *BasicPool* che realizza una coda bloccante
3. Implementare classe *AutoPool* che implementa *Pool* realizzando un meccanismo di *auto – release*. Se *p* è una pool deve essere rilasciata tramite *p.release(x)*. TIP: definire parametro *Resource*
4. Automatizzare il punto sopra facendo in modo che un oggetto *Resource* si rilasci automaticamente: TIP: La classe *Object* definisce il metodo *finalize()* chiamato quando interviene il garbage collector.

```

1 public class Es2 {
2     public interface Pool<T, R> {
3         void add(T x);
4         R acquire() throws InterruptedException;
5         void release(R x);
6     }
7     // ----- 1 -----
8     public interface BasicPool<T> extends Pool<T, T> {}
9     // ----- 2 -----
10    public static class SimplePool<T> implements BasicPool<T> {
11        private BlockingQueue<T> q = new LinkedBlockingDeque<>();
12        @Override
13        public void add(T x) {
14            q.add(x);
15        }
16        @Override
17        public T acquire() throws InterruptedException {
18            return q.take();
19        }
20        @Override
21        public void release(T x) {
22            add(x);
23        }
24    }
25    // ----- 3 -----
26    public interface Resource<T> {
27        T get();
28        void release();
29    }
30    public static class AutoPool<T> implements Pool<T, Resource<T>> {
31        private BlockingQueue<T> q = new LinkedBlockingDeque<>();
32        public void add(T x) {q.add(x);}
33        public Resource<T> acquire() throws InterruptedException {
34            T r = q.take();
35            return new Resource<>() {
36                @Override
37                public T get() {
38                    System.out.println(String.format("acquired:_%s", r));
39                    return r;
40                }
41                @Override
42                public void release() {
43                    System.out.println(String.format("released:_%s", r));
44                    add(r);
45                }
46                // ----- 4 -----
47                @SuppressWarnings("deprecation")
48                @Override
49                public void finalize() {
50                    release();
51                }
52            };
53        }
54        @Override
55        public void release(Resource<T> x) {
56            x.release();
57        }
58    }
59 }

```

Math

Esame 30/6/2023:

1. definire classe *Pair* parametrica su A e B
2. implementare la classe *FunSeq* che rappresenta una *funzione* che itera da *X* a *Y* con type parameter *X* vincolato ad essere *Number* e implementare *Comparable < X >*
3. rappresentare una parabola tramite *lamda* da *R* a *R*
4. rappresentare la stessa parabola con *lambda* ma da *R* a *Z*

```

1 // ----- 1 -----
2 public static class Pair<A, B> {
3     public final A fst;
4     public final B snd;
5     public Pair(A a, B b) {
6         fst = a;

```

```

7         snd = b;
8     }
9 }
10 // ----- 2 -----
11 public static class FunSeq<X extends Number & Comparable<? super X>, Y extends Number>
12     implements Iterable<Pair<X, Y>> {
13     private final X a, b;
14     private final Function<? super X, ? extends Y> f;
15     private final Function<X, X> inc;
16     public FunSeq(X a, X b, Function<? super X, ? extends Y> f, Function<X, X> inc) {
17         this.a = a;
18         this.b = b;
19         this.f = f;
20         this.inc = inc;
21     }
22     @Override
23     public Iterator<Pair<X, Y>> iterator() {
24         return new Iterator<>() {
25             private X x = a;
26             @Override
27             public boolean hasNext() { return x.compareTo(b) <= 0; }
28             @Override
29             public Pair<X, Y> next() {
30                 Pair<X, Y> r = new Pair<>(x, f.apply(x));
31                 x = inc.apply(x);
32                 return r;
33             }
34         };
35     }
36 }
37 // ----- 3 -----
38 for (Pair<Double, Double> p : new FunSeq<>(-2., 2., (x) -> x * x - 2 * x + 1, (x) -> x + 0.1)) {
39     final double x = p.fst, y = p.snd;
40     System.out.printf("f(%g)=%g\n", x, y);
41 }
42 // ----- 4 -----
43 for (Pair<Double, Integer> p : new FunSeq<>(-2., 2., (x) -> (int) (x * x - 2 * x + 1), (x) -> x + 0.1)) {
44     final double x = p.fst;
45     final int y = p.snd;
46     System.out.printf("f(%g)=%d\n", x, y);
47 }

```

Geometry

Esame 31/1/2020:

1. Classe *Point*
2. implementare l'interfaccia *Solid* su metodi statici per il compare e il *compareTo*
3. implementare classe *Cube*. TIP: usare il metodo *move()* di *Point* per calcolare a volo i punti del cubo. Implementare il metodo *at()* usando una anonymous class.
4. implementare classe *Sphere*

```

1 public class Es1 {
2     // ----- 1 -----
3     public static class Point {
4         public final double x, y, z;
5         public Point(double x, double y, double z) {
6             this.x = x;
7             this.y = y;
8             this.z = z;
9         }
10        public Point move(double dx, double dy, double dz) {
11            return new Point(x + dx, y + dy, z + dz);
12        }
13    }
14    // ----- 2 -----
15    public interface Solid extends Comparable<Solid> {
16        double area();
17        double volume();
18        PositionedSolid at(Point origin);
19        static <S extends Solid> int compareBy(Function<S, Double> f, S s1, S s2) {
20            return Double.compare(f.apply(s1), f.apply(s2));
21        }
22        static <S extends Solid> Comparator<S> comparatorBy(Function<S, Double> f) {
23            return (s1, s2) -> compareBy(f, s1, s2);
24        }
25        default int compareTo(Solid s) {
26            return compareBy((x) -> x.volume(), this, s);
27        }
28    }
29    public interface PositionedSolid {
30        Point origin();
31    }
32    public interface Polyhedron extends Solid {
33        double perimeter();
34        @Override
35        PositionedPolyhedron at(Point origin);
36    }
37    public interface PositionedPolyhedron extends PositionedSolid, Iterable<Point> {}

```

```
38 // ----- 3 -----
39 public static class Cube implements Polyhedron {
40     private double side; // lato del cubo
41     public Cube(double side) { this.side = side; }
42     @Override
43     public double area() { return side * side * 6; }
44     @Override
45     public double volume() { return side * side * side; }
46     @Override
47     public double perimeter() { return side * 12; }
48     @Override
49     public PositionedPolyhedron at(Point o) {
50         return new PositionedPolyhedron() {
51             @Override
52             public Point origin() { return o; }
53             @Override
54             public Iterator<Point> iterator() {
55                 final Point u = o.move(side, side, side);
56                 final Point[] ps = new Point[]{
57                     o, o.move(side, 0., 0.), o.move(0., side, 0.), o.move(0., 0., side),
58                     u, u.move(side, 0., 0.), u.move(0., side, 0.), u.move(0., 0., side),
59                 };
60                 return Arrays.asList(ps).iterator();
61             }
62         };
63     }
64 }
65 // ----- 4 -----
66 public static class Sphere implements Solid {
67     private double ray; // raggio della sfera
68     public Sphere(double ray) { this.ray = ray; }
69     @Override
70     public double area() { return 4. * Math.PI * ray * ray; }
71     @Override
72     public double volume() { return 4. / 3. * Math.PI * ray * ray * ray; }
73     @Override
74     public PositionedSolid at(Point o) {
75         // return () -> o;
76         return new PositionedSolid() {
77             @Override
78             public Point origin() {
79                 return o;
80             }
81         };
82     }
83 }
84 }
```

C++

Documentazione

Member Type

Tipo	Descrizione
value_type	Tipo degli elementi contenuti nel contenitore
size_type	Tipo utilizzato per indicizzare e rappresentare dimensioni (solitamente unsigned int)
reference	Tipo riferimento a value_type, equivalente a value_type&
const_reference	Tipo costante riferimento a value_type, equivalente a const value_type&
key_type	Tipo della chiave nei contenitori associativi
mapped_type	Tipo del valore mappato nei contenitori associativi
iterator	Tipo di iteratore che permette di accedere e modificare gli elementi del contenitore
const_iterator	Tipo di iteratore costante che permette di accedere agli elementi del contenitore in modo non modificabile
reverse_iterator	Tipo di iteratore che scorre il contenitore in ordine inverso
const_reverse_iterator	Tipo di iteratore costante che scorre il contenitore in ordine inverso in modo non modificabile

Vector

Member functions	
(constructor)	Costruisce un vettore vuoto.
(destructor)	Distrugge il vettore.
operator=	Assegna valori al contenitore.
assign()	Assegna valori al contenitore.
assign_range()	(C++23) Assegna una gamma di valori al contenitore.
get_allocator()	Restituisce l'allocatore associato al vettore.
Element Access	

at() operator[] front() back() data()	Accede all'elemento specificato con controllo dei limiti. Accede all'elemento specificato. Accede al primo elemento. Accede all'ultimo elemento. Accesso diretto all'array sottostante.
Iterators	
begin() cbegin() end() cend() rbegin() crbegin() rend() crend()	Restituisce un iteratore all'inizio del vettore. (C++11) Restituisce un const_iterator all'inizio del vettore. Restituisce un iteratore alla fine del vettore. (C++11) Restituisce un const_iterator alla fine del vettore. Restituisce un reverse_iterator all'inizio del vettore (versione inversa). (C++11) Restituisce un const_reverse_iterator all'inizio del vettore (versione inversa). Restituisce un reverse_iterator alla fine del vettore (versione inversa). (C++11) Restituisce un const_reverse_iterator alla fine del vettore (versione inversa).
Capacity	
empty() size() max_size() reserve() capacity() shrink_to_fit()	Verifica se il vettore è vuoto. Restituisce il numero di elementi nel vettore. Restituisce il massimo numero possibile di elementi nel vettore. Prenota memoria. Restituisce il numero di elementi che possono essere contenuti nella memoria attualmente allocata. (DR*) Riduce l'uso di memoria liberando la memoria inutilizzata.
Modifiers	
clear() insert() insert_range() emplace() erase() push_back() emplace_back() append_range() pop_back() resize() swap()	Cancella i contenuti del vettore. Inserisce elementi. (C++23) Inserisce una gamma di elementi. (C++11) Costruisce un elemento in loco. Cancella elementi. Aggiunge un elemento alla fine. (C++11) Costruisce un elemento in loco alla fine. (C++23) Aggiunge una gamma di elementi alla fine. Rimuove l'ultimo elemento. Modifica il numero di elementi memorizzati. Scambia i contenuti.

Stack

Member functions	
(constructor)	Costruisce uno stack vuoto.
Capacity	
empty() size()	Restituisce true se lo stack è vuoto, altrimenti false . Restituisce il numero di elementi nello stack.
Element Access	
top()	Restituisce una referenza all'elemento in cima allo stack.
Modifiers	
push() emplace() pop()	Inserisce un elemento in cima allo stack. Costruisce e inserisce un elemento in cima allo stack. Rimuove l'elemento in cima allo stack.
Non-member functions	
swap()	Scambia il contenuto di due stack.

Queue

Member functions	
(constructor)	Costruisce una coda vuota.
(destructor)	Distrugge la coda.
operator=	Assegna valori all'adattatore del contenitore.
Element Access	
front() back()	Restituisce una referenza al primo elemento della coda. Restituisce una referenza all'ultimo elemento della coda.
Capacity	
empty() size()	Verifica se il contenitore sottostante è vuoto. Restituisce il numero di elementi nella coda.

Modifiers	
push()	Inserisce un elemento alla fine della coda.
push_range()	(C++23) Inserisce una gamma di elementi alla fine della coda.
emplace()	(C++11) Costruisce un elemento direttamente alla fine della coda.
pop()	Rimuove il primo elemento dalla coda.
swap()	(C++11) Scambia i contenuti tra due code.

Priority Queue

Element Access	
top()	Restituisce una referenza all'elemento in cima alla coda a priorità.
Capacity	
empty()	Verifica se il contenitore sottostante è vuoto.
size()	Restituisce il numero di elementi nella coda a priorità.
Modifiers	
push()	Inserisce un elemento nella coda e ordina il contenitore sottostante.
push_range()	(C++23) Inserisce una gamma di elementi nella coda e ordina il contenitore sottostante.
emplace()	(C++11) Costruisce un elemento direttamente nella coda e ordina il contenitore sottostante.
pop()	Rimuove l'elemento superiore dalla coda a priorità.
swap()	(C++11) Scambia i contenuti tra due code a priorità.

Map

Element Access	
at()	Accede all'elemento specificato con controllo dei limiti.
operator[]	Accede o inserisce l'elemento specificato.
Iterators	
begin()	Restituisce un iteratore all'inizio della mappa.
cbegin()	(C++11) Restituisce un const_iterator all'inizio della mappa.
end()	Restituisce un iteratore alla fine della mappa.
cend()	(C++11) Restituisce un const_iterator alla fine della mappa.
rbegin()	Restituisce un reverse_iterator all'inizio della mappa (versione inversa).
crbegin()	(C++11) Restituisce un const_reverse_iterator all'inizio della mappa (versione inversa).
rend()	Restituisce un reverse_iterator alla fine della mappa (versione inversa).
crend()	(C++11) Restituisce un const_reverse_iterator alla fine della mappa (versione inversa).
Capacity	
empty()	Verifica se la mappa è vuota.
size()	Restituisce il numero di elementi nella mappa.
max_size()	Restituisce il massimo numero possibile di elementi nella mappa.
Modifiers	
clear()	Cancella i contenuti della mappa.
insert()	Inserisce elementi o nodi (a partire da C++17).
insert_range()	(C++23) Inserisce una gamma di elementi nella mappa.
insert_or_assign()	(C++17) Inserisce un elemento o assegna all'elemento corrente se la chiave esiste già.
emplace()	(C++11) Costruisce un elemento in loco.
emplace_hint()	(C++11) Costruisce elementi in loco utilizzando un suggerimento.
try_emplace()	(C++17) Inserisce in loco se la chiave non esiste, non fa nulla se la chiave esiste.
erase()	Cancella elementi dalla mappa.
swap()	(C++11) Scambia i contenuti tra due mappe.
extract()	(C++17) Estrae nodi dalla mappa.
merge()	(C++17) Unisce nodi da un'altra mappa.
Lookup	
count()	Restituisce il numero di elementi corrispondenti a una chiave specifica.
find()	Trova l'elemento con la chiave specificata.
contains()	(C++20) Verifica se la mappa contiene un elemento con una chiave specifica.
equal_range()	Restituisce l'intervallo di elementi corrispondenti a una chiave specifica.
lower_bound()	Restituisce un iteratore al primo elemento non minore della chiave data.
upper_bound()	Restituisce un iteratore al primo elemento maggiore della chiave data.
Observers	
key_comp()	Restituisce la funzione che confronta le chiavi.
value_comp()	Restituisce la funzione che confronta le chiavi negli oggetti di tipo value_type.

Utils

Lambdas:

```
1 auto f1 = [](int x) { return x + 1; }; // da int a int
2 auto f2 = [](auto x) { return x + 1; }; // auto sul parametro
3 auto f3 = [](auto x) -> int { return x + 1; }; // con annotazione esplicita del tipo di ritorno
4 auto f4 = [](int x) -> int { return x + 1; }; // con annotazione esplicita sul parametro e sul ritorno
5 // altri esempi con reference
6 auto f1 = [](const int& x) { return x + 1; }; // con un const int& come parametro
7 auto f2 = [](const auto& x) { return x + 1; }; // auto usato insieme a const e reference
8 auto f3 = [](auto& x) { x++; }; // come reference non-const
9 // esempi di capture: variabili catturate dalla chiusura della lambda
10 int k = 5;
11 vector<int> v{ 1, 2, 3, 4, 5 };
12 auto f1 = [=](int x) { return x + v[0] + k; }; // v e k sono catturate per COPIA
13 auto f2 = [&](int x) { return x + v[0] + k; }; // v e k sono catturate per REFERENCE
14 auto f3 = [=, &v](int x) { return x + v[0] + k; }; // tutto per copia eccetto v per reference
15 auto f4 = [&, k](int x) { return x + v[0] + k; }; // tutto per reference eccetto k per copia
16 auto f5 = [a = v, b = k](int x) { return x + a[0] + b; }; // tutto per copia con rebinding dei nomi
17 auto f6 = [&a = v, b = k](int x) { return x + a[0] + b; }; // v si chiama a ed per reference; k si chiama b ed per copia
```

Macros:

```
1 namespace macros {
2     void myswap_int(int* a, int* b) {
3         int tmp = *a;
4         *a = *b;
5         *b = tmp;
6     }
7     void myswap_double(double* a, double* b) {
8         double tmp = *a;
9         *a = *b;
10        *b = tmp;
11    }
12    // questa macro si chiama SWAP ed ha un parametro di nome T
13    // l'operatore ## permette di appendere l'argomento della macro senza introdurre spazi
14    // si badi che quando si definisce una macro che deve essere indentata dentro un blocco, la convenzione e'
15    // mettere il # ad inizio riga e tabbare la define
16    // il backslash immediatamente seguito dalla fine della riga, permette di definire macro su piu' righe
17    #define SWAP(T) \
18    void swap_##T(T* a, T* b) { \
19        T tmp = *a; \
20        *a = *b; \
21        *b = tmp; \
22    }
23    // qui usiamo la macro piu' volte: essa definira' varie versioni della swap con nomi diversi: swap_int,
24    // swap_double
25    SWAP(int)
26    SWAP(double)
27    SWAP(char)
28    SWAP(long)
29    export void test(){
30        int x = 3, y = 6;
31        swap_int(&x, &y);
32    }
33 }
```

Ereditarietà:

```
1 export namespace zoo{
2     export class animal{
3     protected:
4         int weight_;
5     public:
6         explicit animal(int w) : weight_(w) {}
7         virtual ~animal() {}
8         animal(const animal& a) : weight_(a.weight_) {}
9         animal& operator=(const animal& a){
10             weight_ = a.weight_;
11             return *this;
12         }
13         virtual void eat(const animal* a) {
14             weight_ += a->weight_;
15         }
16         const int& weight() const { return weight_; }
17         int& weight() { return weight_; }
18     };
19     class dog : public animal{
20     private:
21         int* a; // supponiamo che un dog abbia un array al suo interno (non lo usiamo davvero, e' solo un esempio
22         // per mostrare una new ed una delete)
23     public:
24         explicit dog(int w) : animal(w), a(new int[10]) {} // l'array lo inizializziamo con una new[]
25         ~dog() override{
26             delete[] a; // e' necessario distruggere l'array con una delete[]
27         }
28         void eat(const animal* a) override{
29             weight() = a->weight() * 2;
30         }
31     };
32     class cat : public animal{
33     public:
34         explicit cat(int w) : animal(w) {}
35         void eat(const animal* a) override{
36             weight() = a->weight() / 3;
37         }
38     }
39 }
```

```

37 };
38 class labrador : public dog{
39 public:
40     explicit labrador(int w) : dog(w) {}
41     void eat(const animal* a) override{
42         weight() = a->weight() * 5;
43     }
44 };
45 export void test(){
46     animal fido(50); // questo oggetto e' nello stack
47     animal* pluto = new dog(40); // questo invece e' nello heap, inoltre c'e' subsumption qui
48     fido.eat(pluto); // invoca la eat() di animal
49     pluto->eat(&fido); // invoca la eat() di dog perche' c'e' il dynamic dispatching in azione, grazie al
        fatto che eat() e' virtual
50     animal pluto2 = labrador(40); // questa non e' una subsumption, e' una invocazione del copy-constructor
51     pluto2.eat(pluto); // invoca la eat() di animal semplicemente perche' l'oggetto e' davvero un animal
        copiato da un labrador: nessun dynamic dispatching ha luogo
52     delete pluto; // distruggi pluto chiamando il distruttore di dog: anche il distruttore e' virtual, quindi
        e' in virtual table, quindi e' soggetto a dynamic dispatching pure lui
53 }
54 }

```

Sum

```

1 // questa versione della sum templatizza l'intero container
2 template <class C>
3 typename C::value_type sum(const C& v){
4     if (v.size() > 0){
5         typename C::value_type r(v[0]);
6         for (size_t i = 0; i < v.size(); ++i){
7             typename C::const_reference x(v[i]);
8             r += x;
9         }
10        return r;
11    }
12    return C::value_type();
13 }
14 // questa versione della sum funziona sugli iteratori, naturalmente templatizzati
15 // assume l'esistenza del value_type, quindi non dovrebbe funzionare con i pointer a meno di usare i traits
16 template <class InputIterator>
17 typename std::iterator_traits<InputIterator>::value_type sum(InputIterator first, InputIterator last){
18     // iterator_traits<T> funziona come una type function: crea un tipo con tutti i member type che lo rendono
        compatibile con un iteratore
19     // se gli si passa un tipo che e' gia' un iteratore, non fa niente; se gli si passa un pointer crea tutti i member
        type opportuni
20     typename std::iterator_traits<InputIterator>::value_type r(*first);
21     while (first != last) r += *first++;
22     return r;
23 }
24 // in C++11 e' possibile usare auto
25 template <class InputIterator>
26 auto sum_cxx11(InputIterator first, InputIterator last){
27     auto r(*first);
28     while (first != last) r += *first++;
29     return r;
30 }
31 // questa versione della sum ha 3 parametri: il terzo parametro e' una funzione, il cui tipo e' templatizzato, e
32 template <class InputIterator, class BinFun>
33 auto sum(InputIterator first, InputIterator last, BinFun f){
34     auto r(*first);
35     while (first != last) r = f(r, *first++); // f deve supportare la sintassi di CHIAMATA A FUNZIONE con 2 argomenti
36     return r;
37 }
38
39 // questa classe definisce un overload di operator(), quindi permette alle sue istanze di supportare la sintassi della
        chiamata a funzione
40 class my_binary_function_object{
41 private:
42     double k;
43 public:
44     explicit my_binary_function_object(double k_) : k(k_) {}
45     // questo e' l'operatore che permette alle istanze di questa classe di essere usate COME SE FOSSE UNA FUNZIONE che
        prende 2 argomenti di tipo e ritorna un int
46     double operator()(double a, double b) const {
47         return a + b + k; // somma i 2 argomenti e il campo
48     }
49     // questo metodo statico non serve a niente, se non a mostrare un piccolo test di questa classe per capire come
        funziona
50     static void test(){
51         my_binary_function_object f(7.1); // costruisci un oggetto invocando l'unico costruttore che c'e'
52         double x = f(11.23, 35.0); // la variabile f puo' essere usata COME SE FOSSE UNA FUNZIONE, cioe' applicando 2
            argomenti alla sua destra tra parentesi tonde
53     }
54 };
55 int my_global_binary_function(char a, char b) { return a + b; }
56 export void test(){
57     std::vector<int> v1 { 1, 2, 3, -4, 5 };
58     std::vector<double> v2 { 56.67, 0.0, 4.5, 98.2134 };
59     char a[4] { 11, 24, 123, -23 };
60     // alcuni test delle funzioni sum
61     int x = sum(v1);
62     double y = sum(v2);
63     // con gli iteratori
64     int x = sum(v1.begin(), v1.end());
65     double y = sum(v2.begin(), v2.end());
66     // con i pointer
67     char x = sum(a, a + 10);
68     double b[5];

```

```

69     double y = sum(b, b + 5);
70     // testiamo ora la versione della sum che prende 3 argomenti, di cui il terzo deve supportare la sintassi di chiamata
       a funzione
71     // ci sono varie espressioni possibili che supportano la sintassi di chiamata funzione e che sono quindi compatibili
       con la sum in questione
72     int x = sum(v1.begin(), v1.end(), [](int a, int b) { return a + b; }); // una lambda binaria che la fa somma degli
       argomenti di tipo int
73     int y = sum(v2.begin(), v2.end(), my_binary_function_object(8.67)); // una istanza di un oggetto che supporta la
       sintassi di chiamata con 2 argomenti
74     int z = sum(a, a + 10, my_global_binary_function); // un puntatore ad una funzione globale
75 }

```

Esami

Math

Esame 13/01/2023:

```

1  using real = double;
2  using unary_fun = function<real(const real&)>;
3  #define RESOLUTION (10)
4  class curve{
5  private:
6      real a, b;
7      unary_fun f;
8  public:
9      curve(const real& a_, const real& b_, const unary_fun& f_) : f(f_), a(a_), b(b_) {}
10     real get_dx() const { return (b - a) / RESOLUTION; }
11     pair<real, real> interval() const { return pair<real, real>(a, b); }
12     curve derivative() const{
13         return curve(a, b, [&, dx = get_dx()](const real& x) { // anche la capture [=] sarebbe stata sufficiente
14             const real dy = f(x + dx) - f(x);
15             return dy / dx;
16         });
17     }
18     curve primitive() const{
19         return curve(a, b, [&, dx = get_dx()](const real& x) { // oppure la [&]
20             const real y = f(x);
21             return y * dx;
22         });
23     }
24     real integral() const{
25         const unary_fun& F = primitive();
26         return F(b) - F(a);
27     }
28     real operator()(const real& x) const{return f(x);}
29     class iterator{
30     private:
31         const curve& c;
32         real x;
33     public:
34         iterator(const curve& c_, const real& x_) : c(c_), x(x_) {}
35         pair<real, real> operator*() const{return pair<real, real>(x, c.f(x));}
36         iterator& operator++()// il pre-incremento modifica s stesso e non ritorna una copia{
37             x += c.get_dx();
38             return *this;
39         }
40         iterator operator++(int)// il post-incremento fa una copia, modifica s stesso e ritorna la copia{
41             auto r(*this);
42             ++(*this);
43             return r;
44         }
45         bool operator!=(const iterator& it) const{
46             return fabs(x - it.x) >= c.get_dx();// non si confrontano mai i float direttamente con l'
               operatore di uguaglianza o disuguaglianza
47         }
48     };
49     iterator begin() const{return iterator(*this, a);}
50     iterator end() const{return iterator(*this, b + get_dx());}
51 };
52 ostream& operator<<(ostream& os, const curve& c){
53     os << "dom_=" << c.interval().first << ",_=" << c.interval().second << "]_dx_=" << c.get_dx() << ":_=" << endl;
54     for (curve::iterator it = c.begin(); it != c.end(); ++it){
55         const pair<real, real>& p = *it;
56         const real& x = p.first, & y = p.second;
57         os << "\tf(" << x << ")_=" << y << endl;
58     }
59     return os << endl;
60 }
61 int main(){
62     curve f(-1., 1., [](const real& x) { return x * x; });
63     cout << f << endl
64           << f.derivative() << endl
65           << f.primitive() << endl
66           << f.primitive().derivative() << endl
67           << f.derivative().primitive() << endl
68           ;
69     return 0;
70 }

```

Esame 30/6/2023:

```

1  template <class X, class Y>
2  class fun_seq{

```

```

3 private:
4     const X a, b, dx;
5     const function<Y(const X&)> f;
6 public:
7     using value_type = pair<X, Y>;
8     fun_seq(const X& _a, const X& _b, const X& _dx, const function<Y(const X&)>& _f) : a(_a), b(_b), dx(_dx), f(_f)
9     {}
10    class iterator{
11    private:
12        X x;
13        fun_seq<X, Y> that;
14    public:
15        explicit iterator(const X& _x, const fun_seq<X, Y>& _that) : x(_x), that(_that) {}
16        iterator(const iterator& it) : x(it.x), that(it.that) {}
17        pair<X, Y> operator*() const{ return pair<X, Y>(x, that.f(x));}
18        bool operator!=(const iterator& it){
19            return !(x > it.x - that.dx && x < it.x + that.dx);
20        }
21        iterator operator++(){
22            iterator r(*this);
23            x += that.dx;
24            return r;
25        }
26    };
27    iterator begin() const{return iterator(a, *this);}
28    iterator end() const{return iterator(b, *this);}
29 };
30 export int main(){
31     fun_seq<double, double> sq1(-2., 2., 0.1, [](const double& x) { return x * x + 2 * x - 1; });
32     for (pair<double, double> p : sq1)
33         cout << "f(" << p.first << ")=" << p.second << endl;
34     fun_seq<double, int> sq2(-2., 2., 0.1, [](const double& x) { return int(x * x + 2 * x - 1); });
35     for (pair<double, int> p : sq2)
36         cout << "f(" << p.first << ")=" << p.second << endl;
37     return 0;
38 }

```

Tree

Esame 13/09/2022:

```

1 template <class T>
2 class tree_node{
3 public:
4     T data;
5     tree_node<T>* left, * right;
6     static bool are_equal(const tree_node<T>* a, const tree_node<T>* b){
7         return a == b || (a != nullptr && b != nullptr && *a == *b);
8     }
9 public:
10    tree_node() = default;
11    tree_node(const tree_node<T>& t) = default;
12    tree_node<T>& operator=(const tree_node<T>& t) = default;
13    ~tree_node(){
14        if (left != nullptr){
15            delete left;
16            left = nullptr;
17        }
18        if (right != nullptr){
19            delete right;
20            right = nullptr;
21        }
22    }
23    tree_node(const T& v, tree_node<T>* l, tree_node<T>* r) : data(v), left(l), right(r){ prepopulate();}
24    bool operator==(const tree_node<T>& t) const{
25        return data == t.data && are_equal(left, t.left) && are_equal(right, t.right);
26    }
27    using value_type = T;
28    using const_iterator = typename vector<T>::const_iterator;
29    using iterator = typename vector<T>::iterator;
30 private:
31    // per motivi di semplicita' ogni nodo ha un campo di tipo vector che viene popolato al volo per essere iterator
32    // si faccia attenzione ad un particolare: per poter ritornare begin() ed end() dello stesso vector, bisogna
33    // conservarlo come membro del nodo
34    vector<T> children;
35    void dfs(vector<T>& v){
36        // perche' non popoliamo direttamente il campo children? il motivo e' molto sottile: ogni nodo ha un suo
37        // campo children, ma noi non vogliamo che ogni nodo popoli il proprio vector; noi vogliamo che quando
38        // decidiamo di popolare il campo di children di un certo nodo, allora viene popolato il vector di QUEL
39        // nodo ogni nodo quindi ha un suo campo children che viene popolato con i SUOI sottorami: tutto cio'
40        // e' uno spreco di spazio, certo, pero' permette di fare iteratori a partire da QUALUNQUE nodo in
41        // maniera semplice e immutabile
42        v.push_back(data);
43        if (left != nullptr) left->dfs(v);
44        if (right != nullptr) right->dfs(v);
45    }
46    // questa viene chiamata dal costruttore: ogni nodo prepopola il proprio campo children
47    void prepopulate(){dfs(children);}
48 public:
49    const_iterator begin() const{return children.begin();}
50    const_iterator end() const{return children.end();}
51    iterator begin(){return children.begin();}
52    iterator end(){return children.end();}
53 };
54 // pseudo-costruttori

```

```

49 // invece di fare metodi statici facciamo funzioni templatizzate globali, cosi' il template argument e' inferito e
    diventano piu' comode da usare
50 template <class T>
51 tree_node<T>* lr(const T& v, tree_node<T>* l, tree_node<T>* r){return new tree_node<T>(v, l, r);}
52 template <class T>
53 tree_node<T>* l(const T& v, tree_node<T>* n){return new tree_node<T>(v, n, nullptr);}
54 template <class T>
55 tree_node<T>* r(const T& v, tree_node<T>* n){return new tree_node<T>(v, nullptr, n);}
56 template <class T>
57 tree_node<T>* v(const T& v){return new tree_node<T>(v, nullptr, nullptr);}
58 template <class T>
59 std::ostream& operator<<(std::ostream& os, const tree_node<T>& t){
60     os << t.data;
61     if (t.left != nullptr) os << "(" << *t.left << ")";
62     if (t.right != nullptr) os << "[" << *t.right << "]";
63     return os;
64 }
65 int main(){
66     // usiamo gli shared_ptr per non doverci ricordare di fare delete
67     // con i pseudo-costruttori globali e' comodissimo costruire un albero, basta innestare le chiamate
68     auto t1 =
69         shared_ptr<tree_node<int>>(lr(1,lr(2,v(3),v(4)),r(5,lr(6,v(7),v(8)))));
70     auto t2 = shared_ptr<tree_node<int>>(lr(1,r(5,lr(6,v(7),v(8))),lr(2,v(3),v(4))));
71     // test dell'operatore di stream (<<)
72     cout << "pretty_printer:\n" << endl
73         << "t1:\n" << *t1 << endl// dereferenziamo per stampare perche' il nostro operator<< non vuole un pointer
            ma un reference
74         << "t2:\n" << *t2 << endl;
75     // test dell'operatore di uguaglianza (==)
76     cout << "equality:\n" << (*t1 == *t2) << ",\n" << (*t1->left == *t2->right) << endl;// dereferenziamo gli operandi
        sinistro e destro del nostro operator== perche' non accetta pointer ma reference
77     // test dell'iteratore non-const
78     cout << "iterator:\n";
79     for (tree_node<int>::iterator it = t1->begin(); it != t1->end(); ++it){
80         int& n = *it;// dereferenziamo l'iteratore abbiamo accesso non-const al dato dentro il nodo
81         n *= 2;// il campo data in ogni nodo puo' quindi essere modificato
82         cout << n << "\n";
83     }
84     cout << endl << "t1_modificato:\n" << *t1 << endl; // ristampiamo t1 dopo le modifiche
85     // test dell'iteratore const
86     cout << "const_iterator:\n";
87     for (tree_node<int>::const_iterator it = t1->begin(); it != t1->end(); ++it)// t1->begin() ritorna un iterator,
        che viene convertito in un const_iterator dal costruttore alla linea 142{
88         const int& n = *it; // dereferenziamo l'iteratore abbiamo accesso const al dato dentro il nodo,
            quindi non possiamo modificarlo ma solo leggerlo
89         cout << n << "\n";
90     }
91     cout << endl;
92 }

```

Pair

Esame 1/6/2023:

```

1 template <class A, typename B>
2 class pair{
3     template <class C, typename D> friend class pair; // necessario per il copy constructor templatizzato
4 private:
5     A first;
6     B second;
7 public:
8     pair() : first(), second() {}
9     pair(const A& a, const B& b) : first(a), second(b) {}
10    pair(const pair<A, B>& p) : first(p.first), second(p.second) {}
11    pair<A, B>& operator=(const pair<A, B>& p){
12        first = p.first;
13        second = p.second;
14        return *this;
15    }
16    template <class C, typename D>
17    pair(const pair<C, D>& p) : first(p.first), second(p.second) {}
18    pair<A, B> operator++(int){
19        pair<A, B> tmp(*this);
20        first++;
21        second++;
22        return tmp;
23    }
24    pair<A, B>& operator++(){
25        ++first;
26        ++second;
27        return *this;
28    }
29    bool operator==(const pair<A, B>& p) const{
30        return first == p.first && second == p.second;
31    }
32    bool operator!=(const pair<A, B>& p) const{
33        return !(*this == p);
34    }
35    pair<A, B> operator+(const pair<A, B>& p) const{
36        return pair<A, B>(first + p.first, second + p.second);
37    }
38    pair<A, B>& operator+=(const pair<A, B>& p){
39        first += p.first;
40        second += p.second;
41        return *this;

```

```

42     }
43     const A& fst() const{return first;}
44     A& fst(){return first;}
45     const B& snd() const{return second;}
46     B& snd(){return second;}
47 };

```

Matrix

Esame 3/6/2022:

```

1  class matrix{
2  private:
3      size_t cols;
4      vector<T> v;
5  public:
6      matrix() : cols(0), v() {}
7      matrix(size_t rows, size_t cols_) : cols(cols_), v(rows * cols) {}
8      matrix(size_t rows, size_t cols_, const T& v) : cols(cols_), v(rows* cols, v) {}
9      matrix(const matrix<T>& m) : cols(m.cols), v(m.v) {}
10     typedef T value_type;
11     typedef typename vector<T>::iterator iterator;
12     typedef typename vector<T>::const_iterator const_iterator;
13     matrix<T>& operator=(const matrix<T>& m){
14         v = m.v;
15         return *this;
16     }
17     T& operator()(size_t i, size_t j){return v[i * cols + j];}
18     const T& operator()(size_t i, size_t j) const{
19         return (*this)(i, j);
20     }
21     iterator begin(){return v.begin();}
22     iterator end(){return v.end();}
23     const_iterator begin() const{return begin();}
24     const_iterator end() const{return end();}
25 };
26 int main(){
27     matrix<double> m1; // non inizializzata
28     matrix<double> m2(10, 20); // 10*20 inizializzata col default constructor di double
29     matrix<double> m3(m2); // costruita per copia
30     m1 = m2; // assegnamento
31     m3(3, 1) = 11.23; // operatore di accesso come left-value
32     for (typename matrix<double>::iterator it = m1.begin(); it != m1.end(); ++it) {
33         typename matrix<double>::value_type& x = *it; // de-reference non-const
34         x = m2(0, 2); // operatore di accesso come right-value
35     }
36     matrix<string> ms(5, 4, "ciao"); // 5*4 inizializzata col la stringa passata come terzo argomento
37     for (typename matrix<string>::const_iterator it = ms.begin(); it != ms.end(); ++it)
38         cout << *it; // de-reference const
39 }

```

SmartPointer

Esame 1/7/2022:

```

1  class smart_ptr{
2  private:
3      T* pt;
4      ptrdiff_t offset;
5      bool is_array;
6      using counter_t = unsigned short;
7      counter_t* cnt;
8      void dec(){
9          if (cnt != nullptr && --(*cnt) == 0){
10             if (pt != nullptr) // se non punta a nulla non c'è niente da liberare{
11                 if (is_array) delete pt;
12                 else delete [] pt;
13             }
14             delete cnt;
15         }
16     }
17     void inc(){
18         if (cnt != nullptr) ++(*cnt);
19     }
20 public:
21     using value_type = T;
22     smart_ptr() : pt(nullptr), offset(0), is_array(false), cnt(nullptr) {}
23     explicit smart_ptr(T* pt_, bool is_array_ = false) : pt(pt_), offset(0), is_array(is_array_), cnt(new counter_t(1)) {}
24     smart_ptr(const smart_ptr<T>& p) : pt(p.pt), offset(p.offset), is_array(p.is_array), cnt(p.cnt){inc();}
25     ~smart_ptr(){dec();}
26     smart_ptr<T>& operator=(const smart_ptr<T>& p){
27         dec();
28         pt = p.pt;
29         cnt = p.cnt;
30         offset = p.offset;
31         is_array = p.is_array;
32         inc();
33         return *this;
34     }
35     // subscript
36     // l'operatore di subscript è l'unica implementazione reale; tutti gli altri operatori sono implementati in
    funzione di questo

```

```

37 // questo approccio poco error-prone perch concentra solamente qui il calcolo esatto dell'indirizzo usando l
38 // in altre parole, l'operatore di subscript funge da API interna a basso livello; tutto il resto costruito
39 // sopra di essa
40 const T& operator[](size_t i) const{return pt[offset + i];}
41 T& operator[](size_t i){
42 // capita spesso che l'implementazione const e l'implementazione non-const siano identiche
43 // in questi casi necessario duplicare il codice, che una pratica inelegante ed error-prone
44 // questo trucco sfrutta un giro di const cast per rimandare questa implementazione a quella const appena
45 // sopra, che l'unica che implementiamo davvero
46 // ATTENZIONE: tutto questo NON richiesto dal tema d'esame, lo mostriamo solamente a scopo didattico
47 // per insegnare una tecnica avanzata di non-duplicazione del codice
48 return const_cast<T&>(const_cast<const smart_ptr<T>&>(*this).operator[](i));
49 }
50 // de-reference
51 const T& operator*() const{
52 // usa l'operatore di subscript
53 return pt[0];
54 }
55 T& operator*(){
56 // stessa tecnica per non duplicare: chiamiamo la versione const di questo operatore definita qui sopra,
57 // che l'unica che implementiamo davvero
58 return const_cast<T&>(const_cast<const smart_ptr<T>&>(*this).operator*());
59 }
60 // field access
61 const T* operator->() const{
62 // anche questa implementazione sfrutta altri operatori scritti sopra: in particolare usa de-reference
63 // che a sua volta usa il subscript, in questo modo non richiede manutenzione
64 return &(*this);
65 // si faccia attenzione a un particolare: solo l'operatore * indicato dalla freccetta invoca l'overload
66 // definito da noi
67 // il de-reference di this dentro le parentesi tonde e l'operatore & pi esterno invocano gli operatori
68 // nativi di C++, non i nostri overload
69 }
70 T* operator->(){
71 // altro uso della tecnica avanzata per non duplicare l'implementazione non-const
72 // cerchiamo di capirla: lo scopo chiamare l'implementazione const di questo operatore senza duplicare
73 // il codice
74 // 1) trasformiamo *this (che in questo scope di tipo smart_ptr<T>&) in un const smart_ptr<T>&
75 // 2) ora che *this castato a const, invochiamo l'operatore o il metodo che ci interessa: la
76 // risoluzione dell'overload resolver l'implementazione const, non far una ricorsione!
77 // 3) siccome stiamo invocando la versione const, il risultato const: ma il nostro tipo di ritorno
78 // deve essere un T* non-const, pertanto bisogna const-castare per TOGLIERE il const
79 return const_cast<T*>(const_cast<const smart_ptr<T>&>(*this).operator->());
80 }
81 smart_ptr<T>& operator+=(ptrdiff_t off){
82 offset += off;
83 return *this;
84 }
85 smart_ptr<T> operator+(ptrdiff_t off) const{
86 // questa implementazione usa il copy-constructor e l'operator+= definito qui sopra
87 return smart_ptr<T>(*this) += off;
88 }
89 smart_ptr<T>& operator--(ptrdiff_t off){
90 offset -= off; // analogo al +=
91 return *this;
92 }
93 smart_ptr<T> operator-(ptrdiff_t off){
94 return smart_ptr<T>(*this) -= off;
95 }
96 smart_ptr<T>& operator++(){
97 // questa implementazione usa l'operator+= e basta
98 return *this += 1;
99 }
100 smart_ptr<T>& operator--(){
101 return *this -= 1; // analogo al ++ ma usa il --
102 }
103 smart_ptr<T> operator++(int){
104 smart_ptr<T> r(*this); // copia
105 ++(*this); // pre-incrementa this: usiamo il pre-incremento affinche questa implementazione dipenda
106 // totalmente dall'implementazione di operator++
107 return r; // ritorna la copia
108 }
109 smart_ptr<T> operator--(int){
110 smart_ptr<T> r(*this);
111 --(*this); // analogo a ++
112 return r;
113 }
114 };
115 using std::string;
116 int main()
117 {
118 int* a = new int[5];
119 smart_ptr<int> a1(a, true), a2;
120 a2 = a1 + 10;
121 a1 -= 3;
122 a1 = ++a1 - *a2-- + a1[3];
123 smart_ptr<double> b(new double[10], true);
124 for (unsigned int i = 0; i < 10; ++i)
125 b++;
126 string* sp = new string("ciao");
127 smart_ptr<string> s1(sp), s2;
128 s2 = s1;
129 size_t i = s2->find('a', 0);
130 }

```


Utlis JDK

PairMap:

```
1 public class PairMap<K, V> implements Map<K, V> {
2     private List<Pair<K, V>> l = new ArrayList<>();
3     @Override
4     public Iterator<Pair<K, V>> iterator() {
5         return l.iterator();
6     }
7     @Override
8     public void put(K key, V value) {l.add(new Pair<K, V>(key, value));}
9     @Override
10    public V get(K key) throws KeyNotFoundException {
11        Iterator<Pair<K, V>> it = l.iterator();
12        while (it.hasNext()) {
13            Pair<K, V> p = it.next();
14            if (p.fst.equals(key))return p.snd;
15        }
16        throw new KeyNotFoundException(String.format("key %s not found", key));
17    }
18    @Override
19    public boolean containsKey(K key) {
20        try {
21            get(key);
22            return true;
23        } catch (KeyNotFoundException e) {return false;}
24    }
25    @Override
26    public V removeKey(K key) throws KeyNotFoundException {// TODO per voi}
27 }
```

HashSet:

```
1 public class HashSet<T> implements Set<T> {
2     private List<T> l = new ArrayList<>();
3     @Override
4     public int size() {return l.size();}
5     @Override
6     public void add(T x) {
7         if (!contains(x))l.add(x);
8     }
9     @Override
10    public void remove(T x) {
11        for (int i = 0; i < l.size(); ++i) {
12            T o = l.get(i);
13            if (o.hashCode() == x.hashCode()) l.removeAt(i);
14        }
15    }
16    @Override
17    public boolean contains(T x) {
18        for (int i = 0; i < l.size(); ++i) {
19            T o = l.get(i);
20            if (o.hashCode() == x.hashCode())return true;}
21        return false;
22    }
23    @Override
24    public boolean contains(Predicate<T> p) {return false; //da fare}
25    @Override
26    public Iterator<T> iterator() {return l.iterator();}
27 }
```

HashMap:

```
1 public class HashMap<K, V> implements Map<K, V> {
2     private static int CAPACITY = 1000;
3     private List<List<Pair<K, V>>> l = new ArrayList<>(CAPACITY);
4     @Override
5     public Iterator<Pair<K, V>> iterator() {
6         // TODO fixare o riscrivere per casa
7         return new Iterator<Pair<K, V>>() {
8             private int i = 0, j = -1;
9             private boolean _hasNext = true;
10            @Override
11            public boolean hasNext() {
12                return _hasNext;
13            }
14            @Override
15            public Pair<K, V> next() {
16                Pair<K, V> r = null;
17                if (j >= 0) {
18                    List<Pair<K, V>> inl = l.get(i);
19                    r = inl.get(j++);
20                    if (j > inl.size())j = -1;
21                }
22                else {
23                    for (; i < l.size(); ++i) {
24                        List<Pair<K, V>> inl = l.get(i);
25                        if (inl == null)continue;
26                        r = inl.get(0);
27                        j = 1;
28                    }
29                }
30                if (r == null)_hasNext = false;
31                return r;
32            }
33        };
34    }
```

```

34 }
35 @Override
36 public void put(K key, V value) {
37     int h = key.hashCode() % CAPACITY;
38     List<Pair<K, V>> inl = l.get(h);
39     if (inl == null) {
40         inl = new ArrayList<>();
41         l.set(h, inl);
42     }
43     inl.add(new Pair<>(key, value));
44 }
45 @Override
46 public V get(K key) throws KeyNotFoundException {
47     int h = key.hashCode() % CAPACITY;
48     List<Pair<K, V>> inl = l.get(h);
49     if (inl != null) {
50         for (int i = 0; i < inl.size(); ++i) {
51             Pair<K, V> p = inl.get(i);
52             if (p.fst.equals(key)) return p.snd;}
53     }
54     throw new KeyNotFoundException(String.format("key %s is missing", key));
55 }
56 @Override
57 public boolean containsKey(K key) {// TODO per casa}
58 @Override
59 public V removeKey(K key) throws KeyNotFoundException {// TODO per casa}
60 }

```

Arraylist:

```

1 public class ArrayList<T> implements List<T> {
2     private Object[] a;
3     private int sz;
4     public ArrayList(int capacity) {
5         a = new Object[capacity];
6         sz = 0;
7     }
8     public ArrayList() {this(10);}
9     @Override
10    public int size() {return sz;}
11    @Override
12    public void add(T x) {
13        if (sz >= a.length) {
14            Object[] old = a;
15            a = new Object[old.length * 2];
16            for (int i = 0; i < old.length; ++i)
17                a[i] = old[i];
18        }
19        a[sz++] = x;
20    }
21    // versione in C della removeIf()
22    /*void C_removeIf(int* a, size_t len, bool(*p)(int))
23    {
24        for (int i = 0; i < len; ++i) {
25            int x = a[i];
26            if (p(x)) {
27                removeAt(i);
28            }
29        }
30    }*/
31    private static class MyPredicate implements Predicate<Integer> {
32        @Override
33        public boolean test(Integer x) {
34            return x % 3 == 0;
35        }
36    }
37    public static void main(String[] args) {
38        List<Integer> c = new ArrayList<>();
39        c.add(78);
40        c.add(-456);
41        c.add(7);
42        c.add(4);
43        c.add(7834);
44
45        c.removeIf(new Predicate<Integer>() {
46            @Override
47            public boolean test(Integer x) {return x < 0;}
48        });
49        Function<Integer, Boolean> y = x -> x > 80;
50        c.removeIf(new MyPredicate());
51        c.removeIf(x -> x > 80);
52    }
53    // esempi di method reference
54    public int f(int x) { return x; }
55    public static boolean g(int x, float y) { return true; }
56    public static void main2(String[] args) {
57        BiFunction<ArrayList<Integer>, Integer, Integer> f1 = ArrayList::f;
58        Function<Integer, Integer> f2 = new ArrayList<>():f;
59        BiFunction<Integer, Float, Boolean> g1 = ArrayList::g;
60        BiFunction<Integer, Float, Boolean> g2 = (x, y) -> g(x, y);
61    }
62    @Override
63    public boolean contains(T x) {
64        return contains((T y) -> x.equals(y));
65    }
66    @Override
67    public boolean contains(Predicate<T> p) {
68        Iterator<T> it = iterator();

```

```

69     while (it.hasNext()) {
70         if (p.test(it.next()))return true;
71     }
72     return false;
73 }
74 private class MyIterator implements Iterator<T> {
75     private int pos = 0;
76     @Override
77     public boolean hasNext() {return pos < size();}
78     @Override
79     public T next() {return get(pos++);}
80 }
81 private static class ListIterator<E> implements Iterator<E> {
82     private int pos = 0;
83     private List<E> l;
84     public ListIterator(List<E> l) {this.l = l;}
85     @Override
86     public boolean hasNext() {return pos < l.size();}
87     @Override
88     public E next() {return l.get(pos++);}
89 }
90 @Override
91 public Iterator<T> iterator() {
92     return new Iterator<T>() {
93         private int pos = 0;
94         @Override
95         public boolean hasNext() {return pos < size();}
96         @Override
97         public T next() {return get(pos++);}
98     };
99 }
100 @Override
101 public T get(int index) {return (T) a[index];}
102 @Override
103 public void set(int index, T x) {a[index] = x;}
104 }

```